Migrating LiliShop to Angular 22

A Practical Report

Said Roohullah Allem

https://github.com/jahanalem

Table of Contents

Why Migrate at All?
What I Changed
Part 1 — From NgModules to Standalone
Part 2 — From Reactive Forms to Signal Forms
Part 3 — The Migration Commands Explained
Part 4 — Switching from Karma to Vitest
The Best Order to Do This
Learn How to Do This Yourself

This report explains how I updated the **LiliShop** frontend to **Angular 22** and started using Angular's newest ways of doing things. I left the old NgModule setup behind, switched from Reactive Forms to Signal Forms, ran several automatic tools to clean up the code, and replaced the Karma test runner with Vitest.

The goal is simple: explain *what* I did, *why* each step matters, and *which commands* I used — in plain words, so anyone can follow along and do the same on their own project.

Why Migrate at All?

Angular has been slowly moving toward a simpler way of building apps, built around "signals." Angular 22 (released around June 2026) turns several modern features on by **default**: standalone components, zoneless change detection, and `OnPush` change detection. Signal Forms also became ready for real use in this version.

In short: the old way still works, but the new way needs far less extra code and matches where Angular is going. Doing this now keeps the project clean, easier to read, and ready for the future.

What I Changed

Four big things:

1. **Architecture** — Replaced NgModules and routing modules with standalone components and simple function-based routing.
2. **Forms** — Changed Reactive Forms into the new Signal Forms.
3. **Code clean-up** — Ran several official Angular tools to use the new control flow, the `inject()` function, signal inputs/outputs, and signal-based queries.
4. **Testing** — Moved from Karma to Vitest. (I only converted the tests that already existed — most of them were created automatically — and did not write new ones.)

Let me walk through each part.

Part 1 — From NgModules to Standalone

The old world

In older Angular, **every component had to be listed inside an NgModule** before you could use it. A module was a special class that grouped things together — think of it as a box with a label that says "here are the parts inside me, here are the boxes I need, and here is what other boxes can borrow from me." This was useful back in 2016, but over time it became mostly extra work: many module files existed only to keep Angular happy, not because they actually did anything important.

The new world

A **standalone component** lists what it needs by itself, right inside its own `imports` array. There is no module in the middle. In Angular 22 this is the default — you don't even have to write `standalone: true` anymore.

The easiest way to see the difference is side by side. Here is the same shop feature, before and after.

Before (module way — 4 files):

```
// shop.component.ts
@Component({ selector: 'app-shop', templateUrl: './shop.component.html' })
export class ShopComponent {}

// shop.module.ts
@NgModule({
  declarations: [ShopComponent],
  imports: [CommonModule, ShopRoutingModule],
})
export class ShopModule {}

// shop-routing.module.ts
@NgModule({
  imports: [RouterModule.forChild([{ path: '', component: ShopComponent }])],
  exports: [RouterModule],
})
export class ShopRoutingModule {}

// app-routing.module.ts (the lazy link)
{ path: 'shop', loadChildren: () => import('./shop/shop.module').then(m => m.ShopModule) }
```

After (standalone way — 2 files):

```
// shop.ts
@Component({ selector: 'app-shop', imports: [CommonModule], templateUrl: './shop.html' })
export class Shop {}

// app.routes.ts (the lazy link — no module, no routing module)
{ path: 'shop', loadComponent: () => import('./shop/shop').then(m => m.Shop) }
```

Four files become two. Two NgModules become zero. The route now points straight at the component. Do this for every feature in LiliShop, and the project loses a lot of files while becoming much easier to read.

What moved where

Old (module-based)	New (standalone, v22)
`@NgModule` with `declarations`	the component itself, standalone by default
Module `imports`	each component's own `imports` array
`bootstrapModule(AppModule)`	`bootstrapApplication(App, appConfig)`
`AppModule.providers`	`app.config.ts` → a list of `provideX()` functions
`AppRoutingModule` + `forRoot`	`provideRouter(routes)`

Old (module-based)	New (standalone, v22)
Routes inside a routing module	a plain `app.routes.ts` array
`loadChildren` → a module	`loadComponent` → a component, or `loadChildren` → a routes array
Guards/resolvers as classes	plain functions using `inject()`

One important point: **the router tools didn't change at all.** `RouterOutlet`, `RouterLink`, `Router`, and `ActivatedRoute` work just like before — standalone components simply import them directly instead of getting them from a module.

Angular 22 defaults worth knowing

- **Standalone is the default** — no `standalone: true` needed.
- **Zoneless change detection** is the default (`provideZonelessChangeDetection()`), so Zone.js is no longer included by default. This works well with signals.
- **`OnPush`** is the default change-detection setting for new components, which makes them a bit faster.
- **`paramsInheritanceStrategy` defaults to `'always'`**, so child routes automatically receive their parent's route parameters.
- **`canMatch` guards now take a third parameter** (`currentSnapshot`).

Part 2 — From Reactive Forms to Signal Forms

The core idea in one sentence

> *In Reactive Forms, your data lives in **two places** — your real data on one side, and a separate `FormGroup` on the other — and you have to keep them matching. In Signal Forms, there is only **one place**: your data is the form.*

Here's an easy picture: Reactive Forms are like a **photocopy** of a paper — you keep editing the copy and must remember to update the original too. Signal Forms are like a **shared Google Doc** — whatever you type is instantly the real version. One source of truth, not two.

Everything for Signal Forms lives in the `@angular/forms/signals` package.

How the shape changes

In Signal Forms, you first describe your data as a plain signal, and the form is built from it. The shape of your data *becomes* the shape of your form — a nested object turns into a nested group, and an array turns into a form array, all by itself.

Reactive (the old way):

```
signupForm = this.fb.group({
  name: ['', Validators.required],
  email: ['', [Validators.required, Validators.email]],
```

```
  address: this.fb.group({ city: [''], street: [''] }),
  skills: this.fb.array<string>([]),
});
```

Signal Forms (the new way):

```
// 1. Your data as a signal
signupModel = signal({
  name: '',
  email: '',
  address: { city: '', street: '' },
  skills: [],
});

// 2. The form + all validation in one place
signupForm = form(this.signupModel, (path) => {
  required(path.name, { message: 'Name is required' });
  required(path.email, { message: 'Email is required' });
  email(path.email, { message: 'Please enter a valid email' });
});
```

No `FormGroup`, no `FormArray`, no getters, no type casting. Just data and a schema.

The template changes too

Reactive Forms used tools like `[formGroup]`, `formControlName`, `formGroupName`, and `formArrayName`. Signal Forms replace all of them with one `[formField]` binding, and you read a field's state by **calling** it like a function:

```
<!-- Reactive -->
<input formControlName="name" />
@if (signupForm.get('name')?.touched && signupForm.get('name')?.invalid) {
  <small>Name is required.</small>
}

<!-- Signal Forms -->
<input [formField]="signupForm.name" />
@if (signupForm.name().touched() && signupForm.name().invalid()) {
  @for (e of signupForm.name().errors(); track e.kind) {
    <small>{{ e.message }}</small>
  }
}
```

You reach nested fields with a simple dot (`signupForm.address.city`), and array items by their number (`signupForm.skills[$index]`) — no special tools needed.

Why the new way is nicer

A few annoying things from Reactive Forms simply go away:

- **Rules that compare two fields now sit next to the field.** A rule like "confirm email must match email" can live right on the `confirmEmail` field with `validate()`, instead of sitting far away on the whole group.
- **Dynamic lists are just normal arrays.** Adding a skill is `model.update(m => ({ ...m, skills: [...m.skills, ''] }))` — plain array work, with no `FormArray`, `push`, or `removeAt`.

- **Async checks are built in.** `validateHttp()` handles the server call, the "checking..." state, and the error message for you. (If you need to call a service or add a delay, `validateAsync()` with `rxResource()` gives you more control.)
- **Reading state is shorter.** `form.email().valid()` instead of `form.get('email')?.valid`.

A quick conversion reference

Concept	Reactive Form	Signal Form
Define the form	`fb.group({ ... })`	`form(model, path => { ... })`
Define the data	controls in the group	a `signal({ ... })` object
Bind a field	`formControlName="name"`	`[formField]="form.name"`
Dynamic list	`FormArray` + getter + casting	a plain array in the model
Cross-field rule	group-level validator	`validate()` / `validateTree()`
Async validation	`AsyncValidatorFn` + RxJS	`validateHttp()` / `validateAsync()`
Is the field valid?	`form.get('email')?.valid`	`form.email().valid()`
Submit	`if (form.valid) { ... }`	`submit(form, async () => { ... })`
Module to import	`ReactiveFormsModule`	`FormField` (and helpers)

Part 3 — The Migration Commands Explained

I used a lot of Angular's official **schematics**. A schematic is just an automatic tool: you run a command, and Angular changes your code for you. Here is what each command I used does.

Tools for architecture and code clean-up

`ng generate @angular/core:standalone` Moves the project away from NgModules. You actually run it **three times**, picking one option each time, in this order:

1. Turn all components, directives, and pipes into standalone ones (and fix the test files).
2. Delete the NgModule classes that nothing needs anymore.
3. Update `main.ts` to use `bootstrapApplication()`.

One thing to watch: the tool **won't delete a module that is still being used**. If your lazy routes still point to a module with `loadChildren → some.module`, that module stays. So you usually have to change those lazy routes into the routes-array style *first*, and then the delete step can remove the modules. If you skip this, the tool often says "Nothing to be done."

`ng generate @angular/core:control-flow` Changes your templates from the old style (`*ngIf`, `*ngFor`, `*ngSwitch`) to the new built-in style (`@if`, `@for`, `@switch`). The new style is part of the template

language itself, reads more naturally, and doesn't need you to import `CommonModule` just to write a condition or a loop.

`ng generate @angular/core:inject` Changes the way you ask Angular for services. Instead of using the constructor, you use the `inject()` function. So `constructor(private api: AccountService) {}` becomes `private api = inject(AccountService)`. It's shorter, it works in places where a constructor can't (like guards and resolvers), and it fits the modern Angular style.

`ng generate @angular/core:signal-input-migration` Changes the old `@Input()` properties into the new `input()` style. Inputs become signals, so you read them by calling them — which matches the rest of the signal-based code.

`ng generate @angular/core:output-migration` Changes the old `@Output()` `EventEmitter` properties into the new `output()` style. Same idea as above, but for the events a component sends out.

`ng generate @angular/core:signal-queries-migration` Changes the old query decorators — `@ViewChild`, `@ViewChildren`, `@ContentChild`, `@ContentChildren` — into the new signal versions (`viewChild()`, `viewChildren()`, and so on). The results become signals, so they fit nicely with the rest of the reactive code.

`ng generate @angular/core:cleanup-unused-imports` A clean-up step. After all the changes above, some imports are no longer used (for example, `CommonModule` once you switch to the new control flow). This tool checks your components and removes the imports that nothing uses anymore.

Part 4 — Switching from Karma to Vitest

Karma was Angular's usual test runner for many years, but it is now retired, and people have moved to faster, more modern tools. **Vitest** is a fast test runner (built on Vite) that many teams now use. I converted the tests that already existed — most of them were created automatically when the components were made — instead of writing new ones.

Here are the commands I used and what each one does.

`npm install --save-dev vitest jsdom` Installs Vitest and **jsdom** as development tools. `jsdom` pretends to be a browser inside Node.js, so the component tests can run without opening a real browser like Karma did.

`ng g @schematics/angular:refactor-jasmine-vitest` Runs Angular's tool that changes your old Jasmine-style test files so they work with Vitest. It updates the test code and settings so your existing tests run on the new runner.

`ng g @schematics/angular:refactor-jasmine-vitest --add-imports` The same tool, but the `--add-imports` flag also adds the Vitest imports (like `describe`, `it`, `expect`) at the top of each test file. Jasmine gave you these automatically; Vitest prefers them written out, so this flag adds them for you.

`npm uninstall karma karma-chrome-launcher karma-coverage karma-jasmine karma-jasmine-html-reporter jasmine-core @types/jasmine` The final clean-up. Once Vitest works and the tests pass, this removes all the old Karma and Jasmine packages so they no longer sit in the project.

> *A quick note on my approach: I only converted the tests that already existed, and most of those were made automatically when the components were created. So this step was about getting the old tests to run on a new, modern tool — not about adding more tests. That's an honest limit worth saying out loud, and a good next step would be to write real, meaningful tests now that Vitest is set up.*

The Best Order to Do This

If you want to do a similar migration, the order matters. Here is a safe order to follow:

1. **Upgrade Angular first**, while the app still works the old way. Keep the *version upgrade* and the *code changes* separate, so if something breaks, it's easy to find out why.
2. **Save a clean starting point** (commit your code).
3. **Run the clean-up tools** (control flow, inject, signal inputs/outputs, signal queries). Build the app and commit after each one.
4. **Change lazy routes** from modules to routes arrays, so the module-removing tool can do its job.
5. **Run the standalone tool** in its three steps.
6. **Remove unused imports.**
7. **Move the tests** to Vitest and remove the Karma/Jasmine packages.
8. **Test the app by hand, not just the build.** A successful build does not prove the app still *works* the same. Click through every page, and watch the Network tab to make sure the lazy parts still load only when needed.

The most important rule: **commit often, in small steps.** If something breaks, you can go back to the last good commit and lose almost nothing.

Learn How to Do This Yourself

If you'd like to learn more and try these migrations on your own project, these two step-by-step guides explain everything in detail:

- **Modules → Standalone (components & routing):** angular-modules-vs-standalone-guide.md
- **Reactive Forms → Signal Forms:** angular-forms-guide.md